Software Seguro

```
root@software-seguro:~# ./start_course.sh
```

Clase 1: Arquitectura Web y Protocolo HTTP

Curso Profesional de Ciberseguridad Web y Ethical Hacking

Objetivos de la Sesión

Comprender los fundamentos técnicos de la comunicación web para auditar sistemas con éxito.

Metodología de la Clase



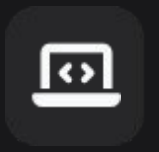
1. Concepto

Estudio de la arquitectura Cliente-Servidor y el protocolo HTTP. Analizaremos cómo fluye la información en la web moderna y su estructura base.



2. Demostración

Resolución guiada del laboratorio "Uso del Inspector". Observaremos en vivo cómo capturar y analizar el tráfico entre el navegador y el servidor.



3. Ejecución

Práctica intensiva en la plataforma Software Seguro. Los alumnos resolverán el laboratorio "Aldeas Inseguras" aplicando los conceptos vistos.

Arquitectura Cliente-Servidor

La web funciona mediante un modelo de comunicación donde un **Cliente** solicita recursos a un **Servidor** a través de internet.

> **El Frontend (Cliente):** Usualmente el navegador web (Chrome, Firefox). Su trabajo es enviar la petición, recibir los datos e interpretarlos (HTML, CSS, JS) para mostrarlos al usuario.

> **El Backend (Servidor):** El equipo remoto que procesa la lógica del negocio, consulta bases de datos y devuelve la información solicitada.

> **Petición y Respuesta:** El cliente inicia la comunicación enviando un *HTTP Request*. El servidor procesa y devuelve un *HTTP Response*.



Anatomía de una URL



Protocolo

Define cómo se transfiere la información (HTTP o versión segura HTTPS).

Subdoinio

División lógica dentro del dominio principal (ej: api, blog, www).

Dominio

El nombre principal registrado que identifica al servidor en internet.

Path Directorio

La ruta o endpoint específico hacia el recurso que queremos acceder.

Parámetros

Variables enviadas al servidor, muy comunes en peticiones GET (clave=valor).

HTTP vs HTTPS

HTTP (Inseguro)

- ✕ **Texto Plano:** Toda la información viaja sin ningún tipo de protección o camuflaje.
  Interceptable: Un atacante en la misma red puede leer contraseñas, correos y tarjetas de crédito fácilmente.
- ✕ **Sin Autenticación:** No hay garantía de que el servidor al que te conectás sea legítimo.

HTTPS (Seguro)

- ✓ **Cifrado (TLS/SSL):** Los datos se transforman en código ilegible antes de viajar por la red.
- ✓ **Privacidad Total:** Aunque un atacante intercepte la conexión (Man-in-the-Middle), solo verá ruido incomprensible.
- ✓ **Identidad Verificada:** Utiliza certificados digitales para asegurar que estás en el sitio real.

Métodos HTTP Fundamentales

`GET` y `POST`

GET: Solicita datos del servidor. Como vimos, los parámetros viajan explícitamente en la URL, por lo que nunca debe usarse para enviar información sensible (contraseñas).

POST: Se utiliza para enviar datos al servidor (ej. enviar un formulario, crear un usuario). La información no viaja en la URL, sino oculta en el "Cuerpo" (Body) de la petición.

`HEAD`, `OPTIONS` y `DELETE`

HEAD: Idéntico a GET, pero el servidor responde únicamente con las cabeceras, sin el cuerpo. Útil para verificar si un recurso existe o fue modificado.

OPTIONS: Solicita al servidor que informe cuáles son los métodos HTTP permitidos en un endpoint particular.

DELETE: Solicita la eliminación de un recurso específico.

Actualización de Recursos: PUT vs PATCH

El Método `PUT`

Su función es **reemplazar** por completo un recurso existente en el servidor.
Si enviamos una petición PUT para actualizar solo el correo de un usuario, y omitimos el campo del nombre, el servidor reemplazará el objeto completo, borrando el nombre que existía previamente (dependiendo de la implementación).

El Método `PATCH`

Su función es realizar una **modificación parcial** del recurso.
A diferencia de PUT, si usamos PATCH para enviar únicamente un nuevo correo electrónico, el servidor actualizará ese campo específico y mantendrá el resto de la información intacta.

Cabeceras (Headers) HTTP

Los Headers son metadatos clave que acompañan a las peticiones y respuestas, brindando contexto extra a la comunicación.

> **Host:**Obligatorio en HTTP/1.1. Indica el dominio exacto al que el cliente desea acceder en el servidor destino.

> **User-Agent:** Cadena de texto que identifica el sistema operativo y navegador del cliente que realiza la solicitud.

> **Cookie:** Permite mantener el "estado" y la sesión del usuario entre diferentes peticiones HTTP, ya que el protocolo por sí mismo no tiene memoria (stateless).

> **Content-Type:** Indica el formato de los datos que se están enviando (ej. JSON, texto plano, datos de formulario).



Familias de Códigos de Estado HTTP

Software Seguro

Familia	Significado General	Ejemplos Comunes en la Web
`2xx (Éxito)`	La petición del cliente fue procesada correctamente.	**200 OK:** Respuesta estándar para peticiones exitosas.
`3xx (Redirección)`	Se requiere una acción adicional para completar la solicitud.	**301 Moved Permanently:** El recurso cambió de URL. **302 Found:** Redirección temporal.
`4xx (Error Cliente)`	Hubo un error del lado del cliente o en los datos enviados.	**401 Unauthorized:** Requiere autenticación. **403 Forbidden:** No hay permisos para acceder. **404 Not Found:** El recurso no existe.
`5xx (Error Servidor)`	El servidor falló al procesar una petición aparentemente válida.	**500 Internal Server Error:** Falla genérica en la lógica del backend.

Preparación del Entorno

Introducción a Burp Suite: El estándar de la industria para auditorías web

Sistema Operativo

Fundamentos y Control

Ubuntu

Ideal para dar los primeros pasos. Ofrece gran estabilidad, documentación masiva y es perfecto para aprender instalando herramientas a mano.

Arch Linux

Para usuarios avanzados. Permite construir el sistema operativo desde cero, logrando un control total sobre cada componente instalado.

Especializados en Pentesting

Kali Linux

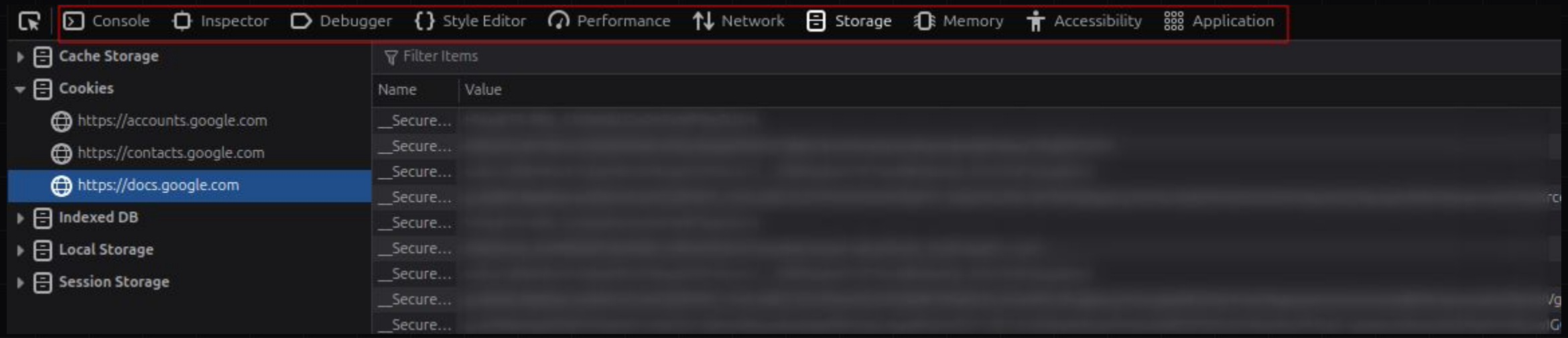
El estándar absoluto de la industria. Viene equipado con cientos de herramientas preinstaladas para auditorías y ethical hacking.

Parrot Security OS

Una alternativa más ligera y amigable con el usuario. Excelente rendimiento y un entorno visual muy cómodo para el uso diario.

Herramientas de Desarrollo

Te permite ver de cualquier sitio web: cómo está construido, qué datos guarda en tu equipo (como las cookies) y exactamente qué información envía y recibe por detrás de escena. En ciberseguridad, es tu primera gran lupa de investigación para espiar el tráfico web y entender cómo se comunican (y cómo se pueden vulnerar) las aplicaciones.



Burp Suite Community Edition / OWASP ZAP

Herramientas que actúan como un **Proxy Web Local** (Man-in-the-Middle). Todo el tráfico entre nuestro navegador y el servidor objetivo pasa a través de él.





Laboratorios de Software Seguro

Pasamos de la teoría a la práctica en plataformas reales

Prácticas de la Sesión (En Clase)

1. Uso del Inspector

Nuestra primera inmersión práctica en el análisis web. Utilizaremos las herramientas de desarrollador nativas del navegador (F12) para capturar peticiones de red, inspeccionar elementos del frontend y observar en tiempo real cómo viajan las cabeceras HTTP que estudiamos.

2. Aldeas Inseguras

Un entorno diseñado para poner a prueba nuestro conocimiento sobre el flujo de peticiones. En este laboratorio, analizaremos las respuestas del servidor para descubrir comportamientos inesperados e interactuar con la aplicación manipulando parámetros específicos.

Asignación para el Hogar

Resolver y enviar los objetivos del Módulo I Introducción al Pentesting Web y Arquitectura HTTP

Ingresar a la plataforma de Software Seguro, en las opciones les aparecera el curso “Curso Profesional de Ciberseguridad Web y Ethical Hacking”, dentro del curso tienen los distintos módulos con sus objetivos que deberán enviar antes de la próxima sesión

Recursos de la Industria

Fundación OWASP

Open Web Application Security Project. La máxima autoridad en estándares de seguridad web. Consulten su "Top 10" para entender las vulnerabilidades más críticas que buscaremos reportar a lo largo del curso.

MDN Web Docs

La documentación de Mozilla es el mejor lugar para consultar dudas sobre el protocolo HTTP, estructura de respuestas, métodos, políticas CORS y estándares oficiales del desarrollo web.

PortSwigger Academy

Los creadores de Burp Suite poseen una academia gratuita con excelente material técnico. Muy recomendable para profundizar en el uso avanzado de cada herramienta dentro de su suite.

¿Preguntas?

Fin de la Sesión 1. Nos vemos en los laboratorios.

https://softwareseguro.com.ar